

FÉDÉRATION FRANÇAISE DES ÉCHECS × CAVALIER DATA

# Un coach IA pour les ouvertures d'échecs

Accompagner les jeunes espoirs pendant leur entraînement autonome

**Richard Hugou**

IA Engineer — Soutenance de projet — Août 2026

# Besoin et contexte

## Le constat à la FFE

- **60 000 licenciés**, dont ~60 % de jeunes espoirs.
- Quelques centaines d'entraîneurs qualifiés sur le territoire.
- **Le goulot d'étranglement est humain** : impossible d'offrir un suivi quotidien individualisé le soir ou entre les rondes de tournoi.

## Le cas d'usage : Léa, 12 ans

- Prépare ses championnats et veut réviser son répertoire ce soir.
- Son entraîneur n'est pas disponible.
- **Mission du POC** : un répétiteur interactif disponible 24/7 pour l'accompagner *pendant* qu'elle joue.

## Les 4 capacités attendues de l'agent

- 1. Jouer** : proposer les coups recommandés par la théorie des maîtres.
- 2. Expliquer** : justifier les plans stratégiques avec sources fiables.
- 3. Conseiller** : recommander des vidéos pédagogiques ciblées.
- 4. Évaluer** : analyser objectivement toute sortie de théorie via Stockfish.

# L'application en action

Chessboard

Tu joues : ▲ les Blancs ▲ les Noirs

Travailler une ouverture :

Italiane Espagnole Sicilienne Française

Caro-Kann Gambit dame Est-indienne Anglaise

Lancer l'IA sur cette position ↶ Annuler le coup

En théorie Italian Game C50

Coups recommandés par la théorie 48,726 parties de maîtres

Fc5 (fou f8)

25,481

Cf6 (cavalier g8)

21,370

Fe7 (fou f8)

1,043

d6 (pion d7)

647

g6 (pion g7)

129

Partie de référence : Caruana, Fabiano – Carlsen, Magnus (2020)

Deux options sérieuses : Fc5 (25 481 parties) ou Cf6 (21 370). La théorie est bien balisée avec des milliers de références. Tu dois choisir entre le fou en c5 et le cavalier en f6 pour continuer la partie italienne Giuoco Piano.

Sources : Base masters — Lichess Opening Explorer (CC0) · Corpus (CC BY-SA) — [https://fr.wikipedia.org/wiki/Partie\\_italienne](https://fr.wikipedia.org/wiki/Partie_italienne) · Corpus (CC BY-SA) — [https://fr.wikipedia.org/wiki/Partie\\_italienne\\_Giuoco\\_Piano](https://fr.wikipedia.org/wiki/Partie_italienne_Giuoco_Piano) · Corpus (CC BY-SA) — [https://en.wikibooks.org/wiki/Chess\\_Opening\\_Theory/1.\\_e4/1...e5/2.\\_Nf3/2...Nc6/3.\\_Bc4](https://en.wikibooks.org/wiki/Chess_Opening_Theory/1._e4/1...e5/2._Nf3/2...Nc6/3._Bc4)

## Démonstration en vidéo

Scénario complet en conditions réelles (~4 min) :

Loom · 3-4 coups · Conseils RAG · Sortie de théorie Stockfish

Cavalier Data x FFE

Interface Angular 17 en direct

3 / 18

# Parcours utilisateur

## 1. Orientation & Choix

L'élève choisit son camp (ex: les Blancs).  
L'échiquier s'oriente immédiatement et  
l'ouverture cible (Partie Italienne) est  
sélectionnée.

## 2. Jeu théorique

L'élève joue ses coups sur le plateau.  
L'agent joue automatiquement la  
réplique des maîtres Lichess la plus  
fréquente (1.e4 e5, 2.Cf3 Cc6).

## 3. Conseils sourcés

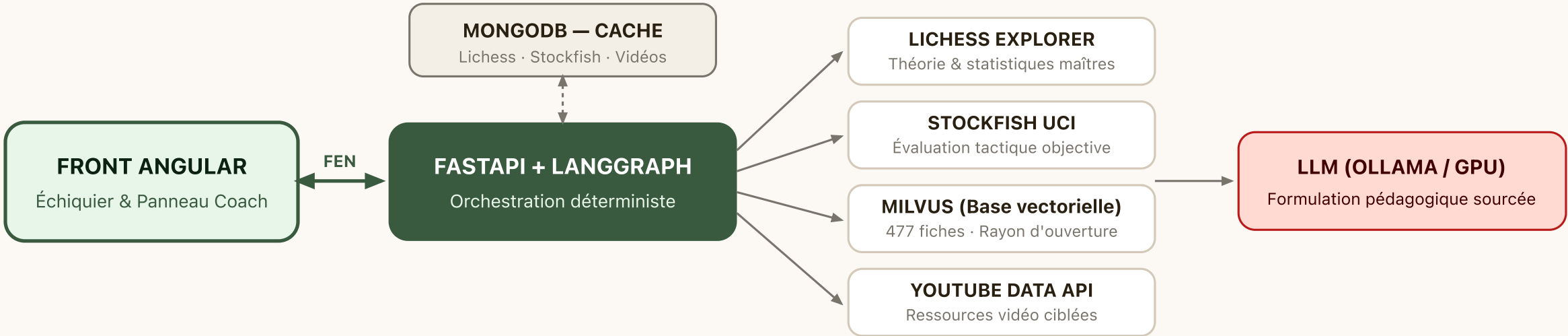
À 3.Fc4, « Demander à Chessbot »  
affiche les coups des maîtres, les  
flèches tactiques, la synthèse sourcée et  
la vidéo YouTube associée.

## 4. Déviation moteur

Sur un coup hors théorie (ex: 4.g4?!),  
l'agent bascule instantanément sur  
Stockfish pour fournir une évaluation  
objective chiffrée (-1,47 cp).

Principe directeur : les faits proviennent des moteurs spécialisés, le LLM intervient uniquement pour la formulation.

# Architecture logicielle



### Clé pivot FEN

La chaîne transmet la position standardisée de bout en bout sans ambiguïté.

### Milvus vs MongoDB

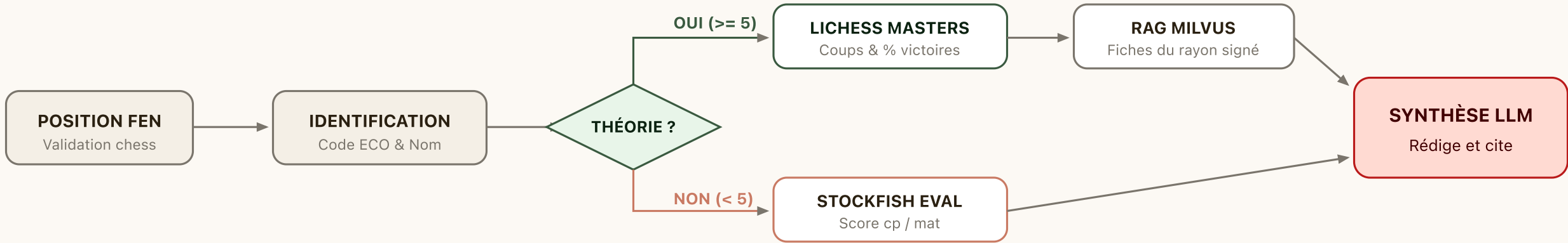
**Milvus** stocke la connaissance vectorisée ; **MongoDB** gère le cache applicatif transverse.

### Routage déterministe

Le seuil de parties maîtres décide du passage théorie/moteur de manière fiable.



# Orchestration de l'agent



## Scénario nominal (Italienne)

Coup 3.Fc4 → Lichess fournit Fc5/Cf6 → Milvus extrait les fiches du rayon → le LLM rédige le plan stratégique.

## Scénario de déviation (Sortie)

Coup 4.g4?! → < 5 parties de référence → bifurcation immédiate vers Stockfish → évaluation -1,47 cp.

# Gisement de données

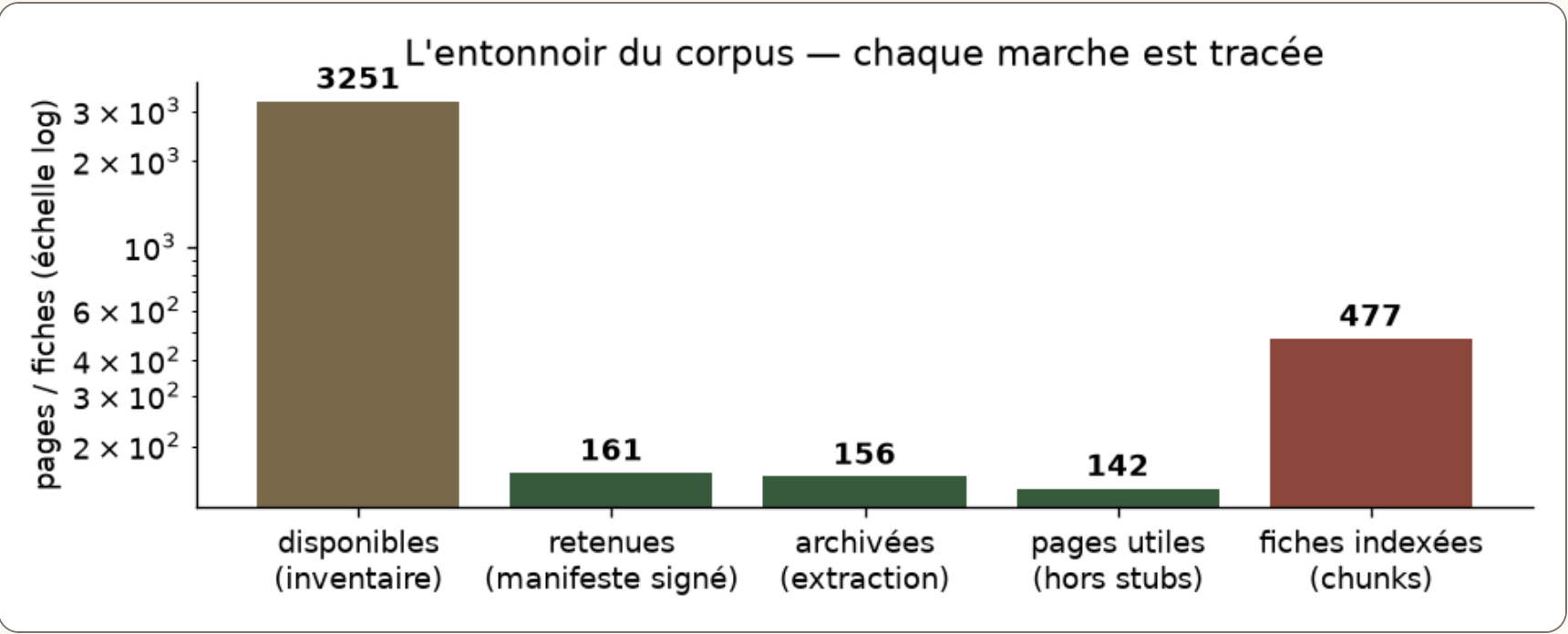
4 sources documentaires complémentaires :

- **Référentiel des ouvertures** : nomenclature et codes ECO officiels
- **Lichess Explorer** : base de 2 M+ parties et statistiques maîtres
- **Wikipédia / Wikibooks** : corpus encyclopédique structuré
- **YouTube** : métadonnées et horodatages des cours vidéo

Pipeline ETL rejouable et idempotent :

extraction (manifeste signé) → nettoyage → fiches → vecteurs → Milvus

Périmètre validé : **156 pages extraites** → **477 fiches structurées** (95 positions FEN calculées, 0 échec).



Entonnoir réel du corpus (notebook 03) : 3 251 pages disponibles → 161 retenues → 477 fiches.

# Recherche sémantique



## Espace vectoriel unifié

Modèle multilingue (1024 dimensions) alignant naturellement les concepts en français et anglais.

## Filtrage par rayon d'ouverture

Recherche restreinte au rayon actif : empêche la récupération de sources hors du périmètre concerné.

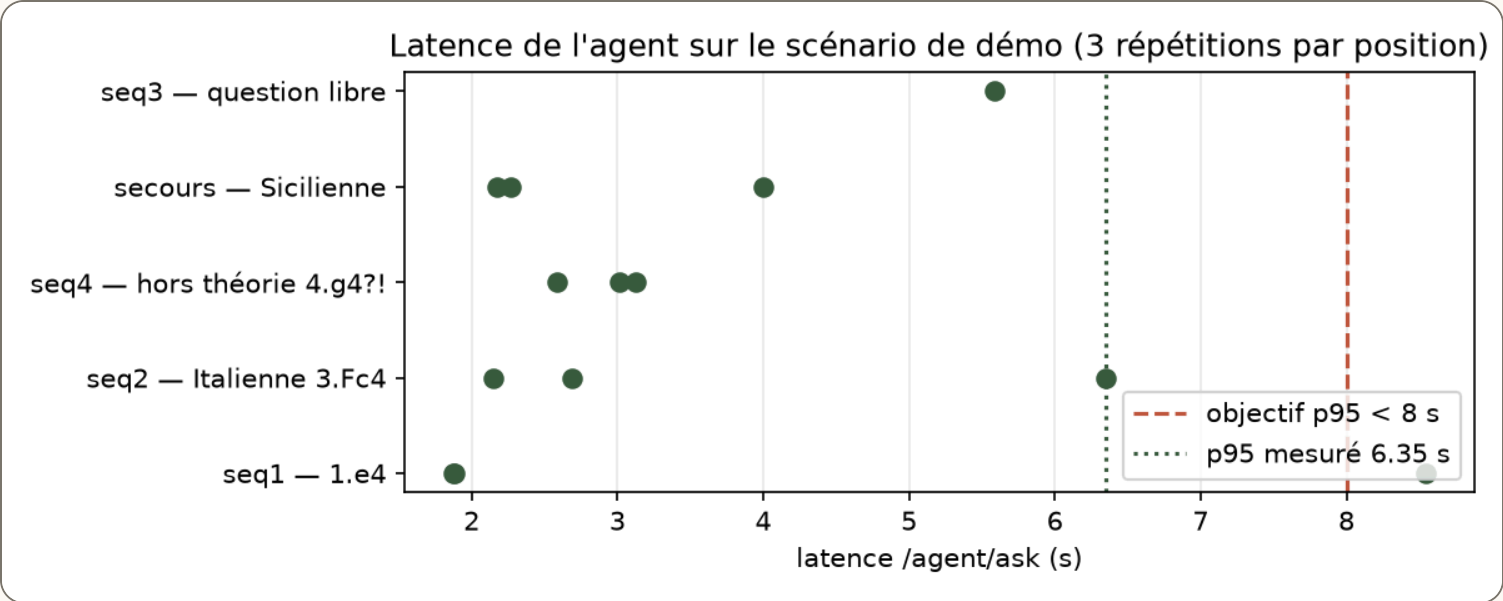
## Performances mesurées

Recherche exécutée en **7 à 11 ms** · Séparation sémantique cible/hors-sujet portée de 0,29 à **0,50**.



# Performances mesurées

| Indicateur clé                   | Objectif  | Résultat mesuré              |
|----------------------------------|-----------|------------------------------|
| Coups illégaux en sortie         | 0         | 0 sur 56 testés              |
| recall@5 (Gold Set 25 questions) | >= 0,80   | 1,0 (tracé MLflow)           |
| Abstention sur les pièges        | 5 / 5     | 5 / 5 (filtrage par rayon)   |
| Réponses avec sources valides    | 100 %     | 100 % (garanti par pipeline) |
| Recherche vectorielle Milvus     | < 100 ms  | 7–11 ms (p95)                |
| Latence Cloud GPU T4 (avec LLM)  | p95 < 8 s | 1,81 s (p50) · 2,41 s (p95)  |
| Latence Local CPU (avec LLM)     | p95 < 8 s | 2,69 s (p50) · 6,35 s (p95)  |
| Coût d'API LLM                   | 0 €       | 0,00 € (modèle open source)  |

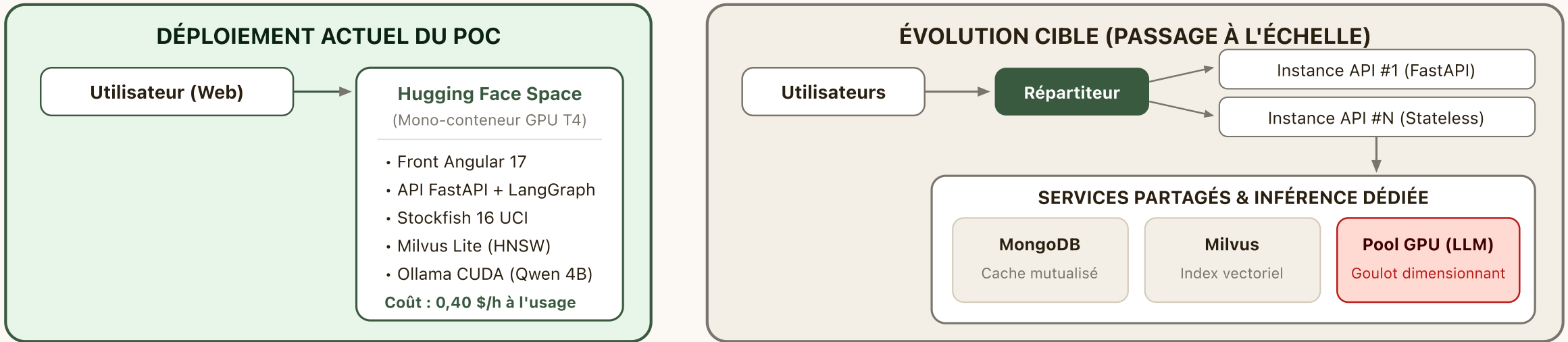


Latences mesurées sur le scénario de démo (notebook 05, 3 répétitions par position).

# Limites du système

| Limite identifiée                                   | Impact sur le système                                           | Mitigation & arbitrage d'ingénierie                                                                                                |
|-----------------------------------------------------|-----------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <b>Dépendances externes</b><br>Lichess, YouTube API | Risque de latence réseau, coupure ou dépassement de quota.      | <b>Cache applicatif transverse MongoDB</b> pour toutes les requêtes fréquentes + mode dégradé gracieux.                            |
| <b>Rôle du LLM</b><br>Risque d'hallucination        | Un LLM peut inventer un coup d'échecs ou une variante tactique. | <b>Aucun choix de coup délégué au LLM</b> : coups imposés par Lichess / Stockfish ; formulation sous contrainte stricte de prompt. |
| <b>Périmètre du POC</b><br>8 ouvertures cibles      | Couverture initiale limitée aux répertoires clés de l'étude.    | <b>Corpus extensible</b> via l'ETL rejouable (et 1/corpus.yml) sans modification du graphe d'orchestration.                        |
| <b>Montée en charge</b><br>Compute GPU              | L'inférence LLM est le facteur limitant en charge simultanée.   | <b>Séparation de tiers</b> : API stateless répliquable + pool d'inférence GPU dimensionné selon la concurrence.                    |

# Déploiement



L'API FastAPI stateless se réplique horizontalement à faible coût ; le pool d'inférence LLM absorbe la charge GPU selon le trafic.

# Dimensionnement et montée en charge

## Facteurs limitants par composant

- **API FastAPI** : Stateless → réplication horizontale simple.
- **Stockfish** : CPU parallélisable (~0,8 s / coup).
- **Milvus & MongoDB** : Services partagés légers (7–11 ms).
- **Inférence LLM : Goulot principal** → dimensionné par le pool GPU.

## Modélisation de volume (Base FFE : 60 000 licenciés)

*Hypothèse d'utilisation : 30 analyses / utilisateur / mois.*

| Scénario      | Utilisateurs | Volume mensuel   | Débit moyen   | Dimensionnement GPU cible   |
|---------------|--------------|------------------|---------------|-----------------------------|
| Scénario 1 %  | 600 users    | 18 000 req/mois  | ~0,25 req/min | Pool GPU partagé            |
| Scénario 5 %  | 3 000 users  | 90 000 req/mois  | ~1,25 req/min | Pool GPU (trafic de pointe) |
| Scénario 10 % | 6 000 users  | 180 000 req/mois | ~2,50 req/min | Pool GPU avec autoscaling   |

*Le dimensionnement GPU réel dépend du volume de concurrence en heure de pointe et du throughput en tokens/s, à valider par test de charge.*

# Coûts de fonctionnement

Estimation budgétaire — Architecture cible de production (Scénario 5 % / 3 000 utilisateurs actifs)

## Postes de dépenses opérationnelles (OPEX mensuel estimé)

| Poste de coût    | Composant                               | Estimation mensuelle |
|------------------|-----------------------------------------|----------------------|
| Inférence GPU    | Pool GPU (instances T4 / L4 à l'usage)  | ~150 à 280 € / mois  |
| API & Compute    | Instances FastAPI stateless (CPU / RAM) | ~30 à 50 € / mois    |
| Bases de données | Milvus managé + MongoDB managé          | ~50 à 80 € / mois    |
| Stockage & Logs  | Métriques, monitoring, sauvegardes      | ~20 € / mois         |
| APIs Externes    | Modèles open source + YouTube API       | 0,00 €               |
| TOTAL OPEX       | Production dimensionnée (3 000 users)   | ~250 à 430 € / mois  |

## Investissement Build (POC)

- **Développement initial POC** : ~25 à 30 jours-homme.
- **Stack 100 % open source** : aucune redevance logicielle propriétaire.
- **Maîtrise des coûts** : infrastructure ajustable à l'usage réel.



# La limite du POC : recommander une vidéo ne suffit pas

Aujourd'hui, l'agent trouve une vidéo par ouverture. L'objectif est de retrouver le passage pertinent.

### Aujourd'hui (POC)

Position de l'élève → Partie Italienne → YouTube Data API → 3 vidéos proposées

- ✗ Mais où est ma position dans les 20 minutes de vidéo ?
- ✗ À quel instant précis est-elle expliquée ?

### Objectif visé (Partie 2)

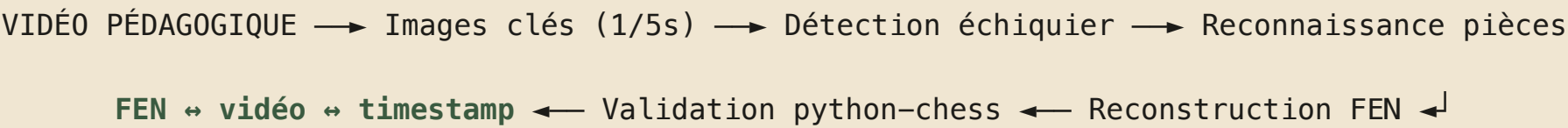
Position de l'élève → FEN → Index des positions vidéo → Vidéo X à 04:32

- ✓ « Cette position exacte est expliquée à 4:32 »
- ✓ Lien direct horodaté pour l'élève.

Le problème à résoudre : passer d'une recommandation par ouverture à une recommandation par position.

# Comment retrouver une position d'échecs dans une vidéo ?

Construire un index : position FEN ↔ vidéo ↔ timestamp



## Exemple d'indexation (Vidéo « L'Italienne »)

- 04:12 → FEN A (Début de variante)
- 04:32 → **FEN B (Position recherchée par l'élève)**
- 05:07 → FEN C (Suite tactique)
- 06:21 → FEN D (Fin d'explication)

## Le FEN comme clé pivot

Les positions extraites des vidéos rejoignent directement la théorie Lichess, Milvus et Stockfish de l'agent existant.

# Faisabilité de l'analyse vidéo

## 1. Approche MVP : Transcripts + Vision 2D

- Reconstruire d'abord les coups cités dans le transcript textuel via python–chess.
- Utiliser la vision par ordinateur 2D uniquement pour confirmer la position et caler l'horodatage (~3 min CPU / vidéo).

## 2. Chiffrage du pilote (100 vidéos)

- **Délai** : 6 à 8 semaines.
- **Effort & Budget Build** : 30 à 40 j-h (15 à 20 k€).
- **Coût Run** : ~0,10 à 0,15 € / vidéo traitée.
- **Cibles** : Précision FEN  $\geq 90\%$ ,  $\geq 30\%$  des requêtes avec timestamp.

## 3. Exclusions du MVP

- **Pas de vision 3D généralisée** ni d'angles inclinés en V1.
- **Périmètre juridique maîtrisé** : vidéos sous licence Creative Commons et chaînes partenaires FFE.

Étude d'ingénierie formalisée (note de cadrage, architecture MCP, modèle de coûts) — Développement hors périmètre du POC.



# Résumé du POC

## Agent fonctionnel

Reconnaissance théorique immédiate, explications pédagogiques sourcées, bascule moteur Stockfish.

## Architecture maîtrisée

Orchestration déterministe LangGraph, Stockfish UCI, base vectorielle Milvus, cache transverse MongoDB.

## Gisement de données

Pipeline ETL rejouable (4 sources, 477 fiches structurées, 95 FEN de référence validés).

## Performances prouvées

0 coup illégal sur 56 positions, latence sous 2,5 s sur GPU Cloud T4 (p95 = 2,41 s).

## Déploiement éprouvé

Conteneurisation reproductible et déploiement GPU validé sur Hugging Face Spaces.

## Étude vidéo livrée

Note de cadrage, architecture MCP, faisabilité technique et modèle de coûts formalisés.

→ Place aux échanges et aux questions du jury.

# Merci.

**Richard Hugou**

IA Engineer — Cavalier Data

Dépôt GitHub public : <https://github.com/richardhugou/p13-agent-ia-echecs>